Module 2

Tables, Data Types, CRUD & Constraints

Turn business rules into a reliable relational schema, then create and change data safely.

BUILD

04

related tables

DDL + DML
with validation

By the end, you can design, populate and protect a schema

1 DESIGN

Choose suitable PostgreSQL types and define primary keys, defaults and relationships.

2 MANIPULATE

Use INSERT, SELECT, UPDATE and DELETE with precise predicates and RETURNING.

3 ENFORCE

Use NOT NULL, UNIQUE, CHECK and FOREIGN KEY constraints to reject invalid state.

Success is visible: invalid rows fail, valid rows persist and every change can be verified.

One build moves from business rules to verified data

01 Model	02 Create	03 Change	04 Hands-on
Translate rules into tables and relationships	Define types, keys and constraints	Practice CRUD with visible results	Build and validate the order schema
15 min	25 min	20 min	30 min

Checkpoint Prove that the schema accepts valid orders and blocks broken ones.

A table is a contract for every stored row

COLUMNS DEFINE MEANING

A name and data type describe each fact: email, price, quantity, status and event time.

KEYS DEFINE IDENTITY

A primary key identifies one row. A foreign key connects that row to a valid parent.

CONSTRAINTS DEFINE LIMITS

Rules reject missing, duplicate, negative or disconnected values before bad data spreads.

Application validation improves usability; database constraints preserve correctness.

Choose the smallest type that preserves the real meaning

BUSINESS VALUE	POSTGRESQL TYPE	WHY IT FITS
Generated identifier	**bigint GENERATED ... AS IDENTITY**	Database creates a stable numeric key
Name, email, SKU	**text**	No arbitrary application-length limit
Money-like amount	**numeric(12,2)**	Exact decimal arithmetic
Quantity	**integer**	Whole-number count
Event time	**timestamptz**	An instant preserved across time zones
Flexible attributes	**jsonb**	Optional structured data

Let PostgreSQL generate identity and routine values

```
customer_id bigint
  GENERATED ALWAYS AS IDENTITY
  PRIMARY KEY,

created_at timestamptz
  NOT NULL DEFAULT now(),

status text
  NOT NULL DEFAULT 'NEW'
```

IDENTITY

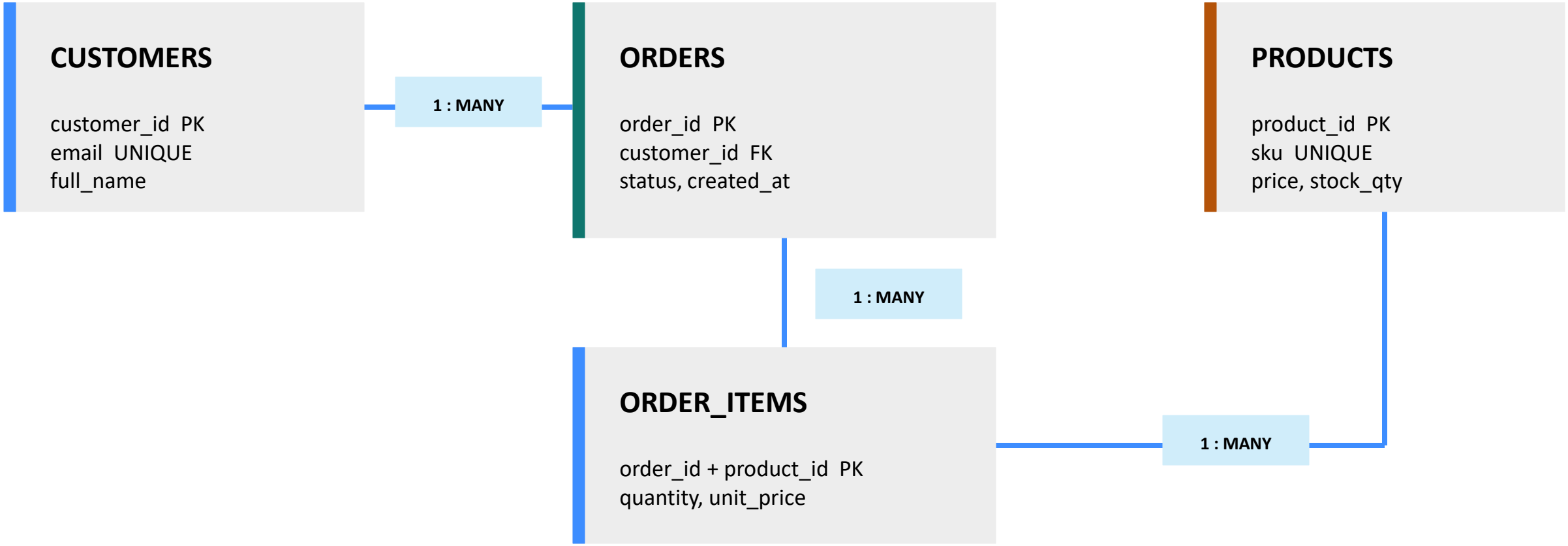
Prefer SQL-standard identity columns over manually calculating the next identifier.

DEFAULT

Use for a meaningful routine value—not to hide a missing required business input.

The database owns technical values; the application supplies business intent.

Four tables capture customers, orders, products and lines



Start with the parent table that owns customer identity

```
CREATE TABLE app.customers (
  customer_id bigint GENERATED ALWAYS AS IDENTITY,
  email       text NOT NULL,
  full_name   text NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT customers_pk PRIMARY KEY (customer_id),
  CONSTRAINT customers_email_uq UNIQUE (email),
  CONSTRAINT customers_email_ck CHECK (position('@' IN email) > 1)
);
```

WHAT IS PROTECTED?

Every row has a name and email. IDs are unique. Duplicate or obviously malformed emails fail.

DESIGN NOTE

The CHECK is a simple training rule—not complete email validation. Real systems also verify ownership.

Product rules prevent impossible prices and stock

```
CREATE TABLE app.products (
  product_id bigint GENERATED ALWAYS AS IDENTITY,
  sku        text NOT NULL,
  name       text NOT NULL,
  price      numeric(12,2) NOT NULL,
  stock_qty  integer NOT NULL DEFAULT 0,
  active     boolean NOT NULL DEFAULT true,
  CONSTRAINT products_pk PRIMARY KEY (product_id),
  CONSTRAINT products_sku_uq UNIQUE (sku),
  CONSTRAINT products_price_ck CHECK (price >= 0),
  CONSTRAINT products_stock_ck CHECK (stock_qty >= 0)
);
```

EXACT PRICE

numeric(12,2) stores an exact decimal value with two fractional digits.

VALID STATE

A product may begin with zero stock, but price and stock can never be negative.

Foreign keys connect orders to valid parent rows

```
CREATE TABLE app.orders (
  order_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id bigint NOT NULL REFERENCES app.customers(customer_id),
  status text NOT NULL DEFAULT 'NEW'
    CHECK (status IN ('NEW','PAID','CANCELLED')),
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE app.order_items (
  order_id bigint REFERENCES app.orders(order_id) ON DELETE CASCADE,
  product_id bigint REFERENCES app.products(product_id),
  quantity integer NOT NULL CHECK (quantity > 0),
  unit_price numeric(12,2) NOT NULL CHECK (unit_price >= 0),
  PRIMARY KEY (order_id, product_id)
);
```

FOREIGN KEY

An item cannot reference an order or product that does not exist.

COMPOSITE KEY

One product appears at most once per order. Deleting an order removes its lines.

Each constraint blocks a different category of bad data

NOT NULL	**Required fact is missing**	full_name cannot be NULL
UNIQUE	**Business identifier is repeated**	email and SKU cannot duplicate
CHECK	**Value breaks a row-level rule**	price ≥ 0; quantity > 0
PRIMARY KEY	**Row identity is missing or repeated**	one stable identifier per row
FOREIGN KEY	**Relationship points nowhere**	order needs a real customer

Inspect the schema before inserting any data

```
\dt app.*
\d app.customers
\d app.products
\d app.orders
\d app.order_items
```

CONFIRM FOUR THINGS

1. All four tables exist
2. Columns use the intended types
3. Constraints have readable names
4. Foreign keys point to the correct parent

If the structure is wrong, fix the DDL now—before test data hides the design mistake.

INSERT with RETURNING makes generated values visible

```
INSERT INTO app.customers (email, full_name)
VALUES ('amira@example.com', 'Amira Hassan')
RETURNING customer_id, email, created_at;

INSERT INTO app.products (sku, name, price, stock_qty)
VALUES
  ('KB-100', 'Mechanical Keyboard', 249.00, 20),
  ('MS-200', 'Wireless Mouse',      89.90, 35)
RETURNING product_id, sku, price;
```

WHY RETURNING?

Capture identity and default values without issuing a second query.

MULTI-ROW INSERT

One statement can add several rows and returns each created result.

Explicit column lists make INSERT statements safer when schemas evolve.

SELECT only the rows and columns the task needs

```
SELECT product_id, sku, name, price, stock_qty
FROM app.products
WHERE active = true
  AND price <= 250
ORDER BY price DESC;
```

READ THE QUERY IN ORDER

FROM chooses the source
WHERE filters rows
SELECT shapes the output
ORDER BY presents the result

Avoid SELECT * in application code: explicit columns communicate intent and reduce accidental coupling.

Preview the predicate before UPDATE changes rows

```
SELECT sku, stock_qty
FROM app.products
WHERE sku = 'KB-100';

UPDATE app.products
SET stock_qty = stock_qty + 10
WHERE sku = 'KB-100'
RETURNING sku, stock_qty;
```

SAFE SEQUENCE

SELECT the target → UPDATE with the same predicate → inspect RETURNING.

DANGER SIGNAL

A missing or broad WHERE clause can update every matching row.

Use an expression such as stock_qty + 10 to change the existing value atomically.

DELETE must respect both intent and relationships

```
SELECT product_id, sku
FROM app.products
WHERE active = false AND stock_qty = 0;

DELETE FROM app.products
WHERE active = false AND stock_qty = 0
RETURNING product_id, sku;
```

RESTRICTED PARENT

A referenced product cannot be deleted while order items still need it.

CASCADE CHILDREN

Deleting an order removes its order_items because that behavior was explicit.

Choose ON DELETE behavior from the business lifecycle—not for convenience.

Hands-on 2: build and test the order schema

A	**Create**	Build customers, products, orders and order_items in app.
B	**Seed**	Insert one customer and at least three products.
C	**Change**	Update stock for one SKU and return the new quantity.
D	**Challenge**	Prove that a negative price and duplicate email both fail.

Keep the failed statements: they are evidence that the constraints work.

A successful lab proves structure, data and protection

1 **Structure**

\dt app.* lists four tables; \d shows keys and constraints.

2 **Valid data**

SELECT shows one customer and at least three products.

3 **Visible change**

UPDATE ... RETURNING displays the new stock quantity.

4 **Rejected state**

CHECK and UNIQUE violations block both invalid attempts.

Stretch: insert a valid order and one item using returned IDs.

Read the SQLSTATE category before changing your code

23505	UNIQUE VIOLATION	The email, SKU or key already exists.
23503	FOREIGN KEY VIOLATION	The referenced parent is missing or still in use.
23514	CHECK VIOLATION	A value breaks a declared business rule.
23502	NOT NULL VIOLATION	A required column received NULL or no value.

Fix the input or model; do not remove a valid constraint to silence the error.

Knowledge check: defend each design choice

1

Why is numeric preferable to floating point for prices?

2

What different problems do PRIMARY KEY and FOREIGN KEY solve?

3

Why should UPDATE and DELETE use RETURNING during a lab?

4

When is ON DELETE CASCADE appropriate—and when is it risky?

Pair-share: support every answer with one table definition or SQL statement.

MODULE 2 COMPLETE

Your schema now rejects invalid business state

DESIGN

Types and keys preserve meaning.

MANIPULATE

CRUD statements make precise, visible changes.

PROTECT

Constraints block missing, duplicate and disconnected data.

Next: Module 3 — Advanced queries, joins and window functions